

Projektdokumentation

Countify

Schulsystem zur Echtzeit-Personenzählung

Filip Mladenovic

1. Projektüberblick

Countify ist ein System zur Echtzeit-Personenzählung in einem Raum. Ein ESP32-Microcontroller mit zwei Infrarot-Sensoren erfasst, wie viele Personen sich im Raum befinden, und sendet diese Daten an einen zentralen Webserver. Dieser speichert die Messwerte und stellt sie über ein Webinterface dar.

Der hier dokumentierte Teil umfasst den Backend-Server (Python/Flask), die Datenbank (SQLite), das Web-Frontend (HTML/JavaScript) sowie die Containerisierung mit Docker.

2. Systemarchitektur

Das System besteht aus drei Hauptkomponenten, die über HTTP miteinander kommunizieren:

- ESP32 (Sensoren): Erfasst Personen per Infrarot und sendet Messwerte per HTTP POST an den Server
- Flask-Webserver (app.py): Empfängt, speichert und liefert Daten über REST-API
- Browser-Frontend (index.html): Zeigt Live-Daten und Verlauf an, kommuniziert mit dem Server

Komponente	Endpoint / Richtung	Protokoll
ESP32 → Server	POST /update	HTTP/JSON
Browser → Server	GET /api/live	HTTP/JSON
Browser → Server	GET /api/verlauf	HTTP/JSON
Browser → ESP32	POST /api/calibrate (direkt)	HTTP/JSON

3. Verwendete Technologien

Komponente	Technologie
Webserver	Python 3.11, Flask
Datenbank	SQLite 3
Frontend	HTML5, CSS3, JavaScript (Vanilla)
Diagramme	Chart.js (CDN)
Containerisierung	Docker, Docker Compose

4. Projektstruktur

```
projekt/
├── app.py                # Flask-Backend (REST API + Datenbanklogik)
├── Dockerfile            # Container-Definition
├── docker-compose.yml    # Container-Orchestrierung
├── requirements.txt       # Python-Abhängigkeiten (flask)
├── templates/
│   └── index.html        # Web-Frontend (Dashboard)
├── data/
│   └── countify.db        # SQLite-Datenbank (persistente
Datenhaltung)
```

5. Backend (app.py)

5.1 Datenbankinitialisierung

Beim Start der Anwendung wird automatisch die SQLite-Datenbank und die Tabelle ‚verlauf‘ angelegt, falls diese noch nicht existieren:

```
from flask import Flask, request, jsonify, render_template
import sqlite3, time, os

app = Flask(__name__)
DB = "/app/data/countify.db"

def get_db():
    con = sqlite3.connect(DB)
    con.row_factory = sqlite3.Row
    return con

def init_db():
    with get_db() as con:
        con.execute("""
            CREATE TABLE IF NOT EXISTS verlauf (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                zeitpunkt      INTEGER NOT NULL,
                personen        INTEGER NOT NULL,
                sensor1        INTEGER,
                sensor2        INTEGER
            )
        """)
```

get_db() öffnet jedes Mal eine neue Datenbankverbindung. Mit con.row_factory = sqlite3.Row können Zeilen wie Dictionaries abgerufen werden. init_db() erstellt die Tabelle verlauf mit fünf Spalten: eine automatisch vergebene ID, ein Unix-Zeitstempel, die Personenanzahl sowie die Rohwerte beider Sensoren.

5.2 Dateneingabe vom ESP32 – /update

Der ESP32 sendet regelmäßig per HTTP-POST die aktuellen Messwerte. Der Server empfängt die JSON-Daten und speichert sie in der Datenbank:

```

@app.route('/update', methods=['GET', 'POST'])
def update():
    if request.method == 'POST':
        d = request.get_json()
        if d is None:
            return jsonify({"error": "kein JSON"}), 400
        with get_db() as con:
            con.execute(
                "INSERT INTO verlauf (zeitpunkt, personen, sensor1,
sensor2) VALUES (?, ?, ?, ?)",
                (int(time.time()), d.get("personen", 0),
                 d.get("sensor1", 0), d.get("sensor2", 0))
            )
            return jsonify({"status": "ok"})
        else:
            with get_db() as con:
                row = con.execute(
                    "SELECT * FROM verlauf ORDER BY zeitpunkt DESC LIMIT 1"
                ).fetchone()
            if row is None:
                return jsonify({"personen": 0, "sensor1": 0, "sensor2": 0})
            return jsonify(dict(row))

```

`int(time.time())` erzeugt einen Unix-Zeitstempel (Sekunden seit 1970) für den aktuellen Moment. Mit `d.get("personen", 0)` werden fehlende Felder sicher mit dem Standardwert 0 behandelt. Fehlt der JSON-Body komplett, wird ein HTTP-Fehler 400 zurückgegeben.

5.3 Live-Daten – /api/live

Liefert den aktuellsten Messwert aus der Datenbank als JSON:

```

@app.route("/api/live")
def live():
    with get_db() as con:
        row = con.execute(
            "SELECT * FROM verlauf ORDER BY zeitpunkt DESC LIMIT 1"
        ).fetchone()
    if row is None:
        return jsonify({"personen": 0, "sensor1": 0, "sensor2": 0})
    return jsonify(dict(row))

```

Gibt immer den zuletzt gespeicherten Datensatz zurück. Falls die Datenbank leer ist (z. B. beim ersten Start), wird ein Standardwert geliefert, damit das Frontend nicht abstürzt.

5.4 Verlaufsdaten – /api/verlauf

Gibt alle Messwerte der letzten 60 Minuten zurück:

```

@app.route("/api/verlauf")
def verlauf():
    seit = int(time.time()) - 3600
    with get_db() as con:
        rows = con.execute(

```

```
"SELECT zeitpunkt, personen FROM verlauf WHERE zeitpunkt > ?
ORDER BY zeitpunkt ASC",
(seit,)
).fetchall()
return jsonify([dict(r) for r in rows])
```

`int(time.time()) - 3600` berechnet den Zeitstempel von vor genau einer Stunde (3600 Sekunden). Die Daten werden chronologisch aufsteigend sortiert, damit das Diagramm im Frontend korrekt dargestellt wird. Das List-Comprehension `[dict(r) for r in rows]` wandelt alle Datenbankzeilen in Python-Dictionaries um.

5.5 Startpunkt

```
if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5000, debug=False)
```

`host="0.0.0.0"` macht den Server im gesamten Netzwerk erreichbar – notwendig für den Zugriff aus dem Docker-Container oder vom ESP32. `init_db()` wird beim direkten Start aufgerufen, um die Datenbank vorzubereiten.

6. Datenbank

6.1 Tabellenstruktur: verlauf

Spalte	Typ	Beschreibung
id	INTEGER (PK, AUTO)	Automatisch vergebene Datensatz-ID
zeitpunkt	INTEGER	Unix-Zeitstempel des Eintrags
personen	INTEGER	Berechnete Personenanzahl im Raum
sensor1	INTEGER	ADC-Rohwert von Sensor 1 (0–4095)
sensor2	INTEGER	ADC-Rohwert von Sensor 2 (0–4095)

Der Sensor-Rohwert liegt im Bereich 0–4095, was dem 12-Bit-ADC des ESP32 entspricht.

6.2 Beispieleinträge aus der realen Datenbank

id	zeitpunkt	personen	sensor1	sensor2
5480	1780663089	0	1899	965
5479	1780663085	0	1744	1083
5478	1780663079	0	1735	937

7. Frontend (index.html)

Das Dashboard wird als Single-Page-Anwendung direkt im Browser ausgeführt und fragt den Server regelmäßig nach neuen Daten ab.

7.1 Aufbau der Seite

Das Frontend besteht aus vier Karten:

- Personenzähler – zeigt die aktuelle Anzahl Personen groß an, mit Live-Status (grüner/roter Punkt)
- Sensor-Rohwerte – zeigt die ADC-Werte von Sensor 1 und Sensor 2 als Fortschrittsbalken
- Verlaufsgraph – Liniendiagramm der letzten 60 Minuten (via Chart.js)
- Kalibrierung – Button zum Auslösen der ESP32-Kalibrierung

7.2 Live-Daten abrufen

```
const MAX = 4095;

async function fetchLive() {
  try {
    const r = await fetch("/api/live");
    const d = await r.json();
    document.getElementById("count").textContent = d.personen;
    document.getElementById("s1val").textContent = d.sensor1;
    document.getElementById("s2val").textContent = d.sensor2;
    document.getElementById("s1bar").style.width = (d.sensor1 / MAX *
100) + "%";
    document.getElementById("s2bar").style.width = (d.sensor2 / MAX *
100) + "%";
    document.getElementById("dot").style.background = "#4CAF50";
    document.getElementById("statusTxt").textContent =
      "Live - " + new Date().toLocaleTimeString("de-AT");
  } catch (e) {
    document.getElementById("dot").style.background = "#f44336";
    document.getElementById("statusTxt").textContent = "Keine
Verbindung";
  }
}

setInterval(fetchLive, 2000); // alle 2 Sekunden aktualisieren
```

`fetch("/api/live")` ruft den Live-Endpunkt des Flask-Servers ab. Die Sensorbalken werden prozentual berechnet: $\text{sensor1} / 4095 * 100 \%$. Bei einem Netzwerkfehler färbt sich der Statusdot rot. `setInterval` sorgt dafür, dass die Anzeige alle 2 Sekunden automatisch aktualisiert wird.

7.3 Verlaufsgraph

```
async function fetchVerlauf() {
```

```

try {
  const r = await fetch("/api/verlauf");
  const data = await r.json();
  chart.data.labels = data.map(d =>
    new Date(d.zeitpunkt * 1000).toLocaleTimeString("de-AT",
      { hour: "2-digit", minute: "2-digit" })
  );
  chart.data.datasets[0].data = data.map(d => d.personen);
  chart.update();
} catch(e) {}

setInterval(fetchVerlauf, 10000); // alle 10 Sekunden aktualisieren

```

Unix-Zeitstempel werden mit * 1000 in JavaScript-Millisekunden umgerechnet. `toLocaleTimeString("de-AT")` gibt die Uhrzeit im österreichischen Format (HH:MM) aus. Das Diagramm wird alle 10 Sekunden aktualisiert, um unnötige Datenbankabfragen zu reduzieren.

7.4 Kalibrierung

```

const ESP32_URL = "http://172.20.10.2";

async function calibrate() {
  const btn = document.getElementById("calBtn");
  btn.disabled = true;
  btn.textContent = "Läuft...";
  try {
    const r = await fetch(ESP32_URL + "/api/calibrate", { method: "POST"
  });
    const d = await r.json();
    if (d.status === "ok") {
      document.getElementById("calStatus").textContent = "✓
Kalibrierung abgeschlossen";
    } else { throw new Error(); }
  } catch(e) {
    document.getElementById("calStatus").textContent = "X ESP32 nicht
erreichbar";
  }
  btn.textContent = "Kalibrierung starten";
  btn.disabled = false;
}

```

Der Browser spricht den ESP32 direkt unter seiner IP-Adresse 172.20.10.2 an. Der Button wird während der Kalibrierung deaktiviert, damit kein doppeltes Auslösen möglich ist. Ist der ESP32 nicht erreichbar, wird eine Fehlermeldung angezeigt.

8. Docker-Containerisierung

8.1 Dockerfile

```

FROM python:3.11-slim

WORKDIR /app

```

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN mkdir -p /app/data

EXPOSE 5000

CMD ["python", "app.py"]
```

python:3.11-slim ist ein schlankes Basis-Image ohne unnötige Pakete. Die Abhängigkeiten (requirements.txt) werden zuerst installiert, bevor der restliche Code kopiert wird – das ermöglicht Docker, diese Schicht zu cachen und spätere Builds zu beschleunigen. Der Ordner /app/data wird angelegt, um die Datenbank persistent speichern zu können.

8.2 Docker Compose

```
version: "3.9"

services:
  countify:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - ./data:/app/data
    restart: unless-stopped
```

ports: "5000:5000" leitet Port 5000 vom Host in den Container weiter. volumes: ./data:/app/data bindet den lokalen data/-Ordner in den Container ein – dadurch bleibt die SQLite-Datenbank auch nach einem Container-Neustart erhalten (persistente Datenhaltung). restart: unless-stopped startet den Container automatisch neu, falls er abstürzt.

9. Deployment – Inbetriebnahme

Voraussetzungen: Docker und Docker Compose müssen installiert sein.

```
# 1. In das Projektverzeichnis wechseln
cd projekt

# 2. Container bauen und starten
docker compose up --build -d

# 3. Im Browser öffnen
# http://<Server-IP>:5000

# Server stoppen:
docker compose down
```


Der Server läuft nach dem Start dauerhaft und startet bei einem Systemneustart automatisch neu (restart: unless-stopped).

10. API-Übersicht

Methode	Endpoint	Beschreibung
GET	/	Liefert das Web-Dashboard
POST	/update	Empfängt Messwerte vom ESP32 (JSON)
GET	/update	Gibt den letzten Messwert zurück
GET	/api/live	Aktueller Messwert (für Frontend)
GET	/api/verlauf	Verlauf der letzten 60 Minuten

Beispiel-Request vom ESP32

```
POST /update
Content-Type: application/json

{
  "personen": 2,
  "sensor1": 1850,
  "sensor2": 920
}
```

Beispiel-Response /api/live

```
{
  "id": 5480,
  "zeitpunkt": 1780663089,
  "personen": 0,
  "sensor1": 1899,
  "sensor2": 965
}
```

Countify – Projektdokumentation

Softwareentwicklung – Muhammed Akkus

1. Mein Aufgabenbereich

Im Projekt Countify war ich für die Projektleitung und die Softwareentwicklung auf dem ESP32 verantwortlich.

Mein Schwerpunkt lag auf der Entwicklung der Personenzählung, der Verarbeitung der Sensordaten, der WLAN-Anbindung sowie verschiedenen Maßnahmen zur Verbesserung der Messgenauigkeit.

2. Personenzählung

Für die Personenzählung werden zwei Infrarot-Sensoren verwendet. Die Bewegungsrichtung wird anhand der Reihenfolge der Sensoraktivierungen erkannt:

- Sensor 2 → Sensor 1 = Person betritt den Raum (+1)
- Sensor 1 → Sensor 2 = Person verlässt den Raum (-1)

Dafür wurde eine Zustandslogik implementiert:

```
enum Zustand {  
    IDLE,  
    S1_ZUERST,  
    S2_ZUERST  
};
```

Dadurch merkt sich das System, welcher Sensor zuerst ausgelöst wurde und wartet anschließend auf die Aktivierung des zweiten Sensors. Wird die erwartete Reihenfolge erkannt, wird der Personenzähler entsprechend erhöht oder verringert.

3. Verbesserung der Messgenauigkeit

Während der Entwicklung kam es teilweise zu schwankenden Sensorwerten. Um Fehlzählungen zu reduzieren, wurden mehrere Maßnahmen umgesetzt.

Mittelwertbildung

```
int leseStabil(int pin) {
```

```

long summe = 0;

for (int i = 0; i < 5; i++) {
    summe += analogRead(pin);

    delay(2);
}

return summe / 5;
}

```

Hier werden fünf Messungen durchgeführt und anschließend gemittelt. Dadurch werden einzelne Fehlmessungen reduziert.

Entprellung

```
const unsigned long ENTPRELLZEIT = 80;
```

Ein Sensor gilt erst dann als aktiv, wenn er für mindestens 80 Millisekunden dauerhaft ausgelöst wurde.

Timeout

```
const unsigned long TIMEOUT_MS = 5000;
```

Falls die zweite Sensorauslösung nicht innerhalb von fünf Sekunden erfolgt, wird der Vorgang verworfen und das System zurückgesetzt.

Diese Maßnahmen verbesserten die Stabilität der Personenzählung deutlich, auch wenn das System nicht in jeder Situation perfekt arbeitet.

4. Versuch einer automatischen Kalibrierung

Zusätzlich wurde ein eigener Kalibrierungsmodus entwickelt. Die Idee war, die aktuellen Umgebungswerte zu messen und daraus automatisch passende Schwellwerte für die Sensoren zu berechnen.

```
const unsigned long KAL_DAUER = 8000;
```

Während der Kalibrierung werden für acht Sekunden Messwerte gesammelt. Anschließend werden daraus neue Schwellwerte berechnet:

```
SENSOR1_SCHWELLWERT =
```

```
s1Ruhe - (int)(s1Ruhe * 0.15f);
```

```
SENSOR2_SCHWELLWERT =
```

```
s2Ruhe - (int)(s2Ruhe * 0.15f);
```

Die Kalibrierung wurde implementiert und getestet, arbeitete jedoch aufgrund unterschiedlicher Lichtverhältnisse und Messschwankungen nicht immer zuverlässig. Deshalb konnte sie nicht vollständig in das Endsystem übernommen werden.

5. Versuch der Erkennung von zwei Personen gleichzeitig

Ein weiteres Ziel war die Erkennung von zwei Personen, die den Eingang nahezu gleichzeitig passieren. Hierfür wurden verschiedene Ansätze getestet und die bestehende Zähllogik erweitert.

In den praktischen Tests zeigte sich jedoch, dass die beiden verwendeten Sensoren nicht genügend Informationen liefern, um zwei nahe beieinander laufende Personen zuverlässig voneinander zu unterscheiden.

Die Funktion konnte daher nicht erfolgreich umgesetzt werden. Die Tests lieferten jedoch wichtige Erkenntnisse für zukünftige Verbesserungen.

6. WLAN-Konfiguration

Für die WLAN-Verbindung wurde die Bibliothek WiFiManager verwendet.

```
WiFiManager wm;
```

```
if (!wm.autoConnect(
```

```
"Countify-Setup",
```

```
"12345678")) {
```

```
ESP.restart();
```

```
}
```

Dadurch müssen WLAN-Zugangsdaten nicht direkt im Quellcode gespeichert werden. Beim ersten Start erstellt das System einen eigenen Access Point mit dem Namen "Countify-Setup". Nach erfolgreicher Einrichtung verbindet sich das Gerät automatisch mit dem gespeicherten WLAN.

7. Lokale Diagnoseoberfläche

Für Tests und Fehlersuche wurde eine einfache lokale Weboberfläche auf dem ESP32 implementiert.

Diese zeigt unter anderem:

- aktuelle Personenzahl
- Sensorwerte
- Schwellwerte
- Kalibrierungsstatus

Zusätzlich kann der Personenzähler über die Oberfläche zurückgesetzt werden:

```
server.on("/reset",
HTTP_GET, []() {
    personCount = 0;
    resetZustand();
});
```

Dadurch konnten Fehler während der Entwicklung schneller erkannt und behoben werden.

8. Fazit

Im Rahmen meines Projektteils konnte eine funktionierende Software zur automatischen Personenzählung entwickelt werden.

Besonders viel Erfahrung konnte ich im Bereich Mikrocontroller-Programmierung, Sensortechnik, Zustandsautomaten und WLAN-Kommunikation sammeln.

Trotz einiger Herausforderungen bei der Kalibrierung und der Erkennung mehrerer Personen gleichzeitig konnte eine stabile Grundlage für das Projekt Countify geschaffen werden.

Hardwareentwicklung und Montage Mahir Kaplan

1. Mein Aufgabenbereich

Im Projekt Countify war ich für den Hardware-Aufbau, die mechanischen Gehäusemodifikationen und die elektronische Verkabelung des Sensormoduls verantwortlich.

Mein Schwerpunkt lag darauf, ein mobiles, autarkes und geschütztes Messsystem (Stand-Alone-System) zu entwickeln, das flexibel im Raum platziert werden kann und die optimale Basis für die Datenerfassung bietet.

2. Verwendete Hardware-Komponenten

Komponente	Beschreibung
ESP32	Zentraler Mikrocontroller zur Datenerfassung und WLAN-Kommunikation.
Sharp GP2Y0A21YK0F	2× Analoge Infrarot-Abstandssensoren zur Erkennung der Personen.
Feuchtraum-Abzweigdose	IP55 Gehäuse (150 × 110 × 70 mm) zum Schutz der Elektronik.
Kamerastativ	Trägersystem für die freistehende und flexible Positionierung im Raum.
Breadboard & Kabel	Male-to-Male Jumperkabel und Steckplatine zur internen Stromverteilung.

3. Mechanischer Aufbau und Gehäusemodifikation

Um das System vor äußeren Einflüssen zu schützen und mobil einsetzbar zu machen, wurde eine handelsübliche IP55-Feuchtraum-Abzweigdose modifiziert.

Sensor-Ausgänge

Auf der Vorderseite des Gehäuses wurden zwei präzise Öffnungen gebohrt. Die Linsen der beiden Infrarot-Sensoren wurden so im Inneren fixiert, dass sie durch diese Löcher nach außen ragen, um eine ungehinderte Distanzmessung zu gewährleisten.

Kabeldurchführung

Für eventuelle externe Stromversorgungen oder Datenleitungen wurden die werkseitig vorhandenen, gummierten Stufenverschraubungen an den Seiten genutzt. So bleibt das Innere vor Staub und Feuchtigkeit geschützt.

Stativmontage

An der Unterseite des Gehäuses wurde eine zentrierte Bohrung vorgenommen, um ein Kamerastativ fest zu verschrauben. Dadurch fungiert das System als eigenständige Stand-Alone-Einheit, die unabhängig von Wänden oder Tischen in der optimalen Höhe am Raumeingang aufgestellt werden kann.

4. Elektrische Verkabelung und Pin-Belegung

Die Sensoren wurden über Male-to-Male Jumperkabel mit dem ESP32 und dem internen Breadboard verbunden. Da die Sharp-Sensoren analoge Signale ausgeben, nutzen wir die entsprechenden Analog-Digital-Wandler-Pins (ADC) des ESP32, um die Rohwerte präzise an die Software zu übergeben.

Signal / Bauteil	ESP32 Pin-Belegung	Beschreibung
Sensor 1 (Analog)	GPIO 4	Erfassung des analogen Spannungswerts von Sensor 1 (ADC).
Sensor 2 (Analog)	GPIO 3	Erfassung des analogen Spannungswerts von Sensor 2 (ADC).
Masse (GND)	GND	Gemeinsames Bezugspotenzial (parallel verteilt über Breadboard).
Stromversorgung	5V (5v5)	5V-Versorgungsspannung für beide Sensoren.